

Notebook 49: Final FMA Processed Test-Split Batch Evaluation

Purpose

This notebook runs the final trained FMA candidate-150 genre classification pipeline against the **processed expanded held-out test split** used by Notebooks 34-36.

It is intended to produce the main formal evaluation evidence for the final capstone documentation.

What this notebook does

1. Loads the final structured model, audio CNN model, label list, and hybrid configuration.
2. Loads FMA metadata, genre hierarchy, engineered features, and audio file paths.
3. Selects the processed expanded candidate-150 test records with valid audio files.
4. Uses the same first-15-second audio preprocessing used during audio CNN training.
5. Runs batch prediction using the final hybrid pipeline.
6. Saves checkpointed predictions so the run can resume if interrupted.
7. Calculates multi-label metrics: Micro F1, Macro F1, Samples F1, precision, recall, Top-K hit-any rates, and per-label metrics.
8. Exports CSV and JSON outputs for the final report, supervisor review, and presentation.

Important note

This notebook is for **formal FMA benchmark evaluation**. Full-song windowed testing of external audio should be handled separately in Notebook 50, because that is a deployment-style inference strategy rather than the training-matched benchmark.

In [1]:

```
# =====  
# Cell 0: Environment Check  
# =====  
  
import sys  
from pathlib import Path  
  
print("Python executable:")  
print(sys.executable)  
print("\nPython version:")  
print(sys.version)  
  
# Adjust this if your project folder name changes.
```

```

EXPECTED_ENV_PART = r"Capstone_FMA_Project"

if EXPECTED_ENV_PART.lower() not in sys.executable.lower() and
EXPECTED_ENV_PART.lower() not in str(Path.cwd()).lower():
    print("\nWARNING: This notebook may not be running inside the
Capstone_FMA_Project environment.")
    print("If TensorFlow or sklearn imports fail, switch the VS Code kernel to your
project .venv.")
else:
    print("\nProject environment/path looks OK.")

```

Python executable:

E:\SCHOOL\Masters\Capstone_FMA_Project\notebook\.venv\Scripts\python.exe

Python version:

3.13.14 (tags/v3.13.14:fd17997, Jun 10 2026, 13:03:48) [MSC v.1944 64 bit (AMD64)]

Project environment/path looks OK.

In [2]:

```

# =====
# Cell 1: Imports
# =====

import os
import ast
import json
import time
import math
import pickle
import warnings
from pathlib import Path
from datetime import datetime

import numpy as np
import pandas as pd

from scipy.special import expit
from sklearn.metrics import (
    precision_recall_fscore_support,
    precision_score,
    recall_score,
    f1_score,
    hamming_loss,
    jaccard_score,
)

try:
    import joblib
except Exception as e:
    raise ImportError("joblib is required to load sklearn model artifacts. Install
with: pip install joblib") from e

try:
    import librosa

```

```

except Exception as e:
    raise ImportError("librosa is required for audio loading and Mel spectrogram
extraction. Install with: pip install librosa") from e

try:
    import tensorflow as tf
    print("TensorFlow version:", tf.__version__)
except Exception as e:
    raise ImportError(
        "TensorFlow could not be imported. Make sure the notebook is using the
correct project .venv kernel."
    ) from e

warnings.filterwarnings("ignore")
print("Imports completed.")

```

TensorFlow version: 2.20.0

Imports completed.

In [3]:

```

# =====
# Cell 2: Configuration
# =====

# Project root. This should match your local Windows project folder.
PROJECT_ROOT = Path(r"E:\SCHOOL\Masters\Capstone_FMA_Project")

# Metadata, processed data, model, and audio locations.
METADATA_DIR = PROJECT_ROOT / "data" / "raw" / "metadata"
PROCESSED_DIR = PROJECT_ROOT / "data" / "processed"
MODELS_DIR = PROJECT_ROOT / "models"
TRACKS_CSV = METADATA_DIR / "tracks.csv"
GENRES_CSV = METADATA_DIR / "genres.csv"
FEATURES_CSV = METADATA_DIR / "features.csv"

# FMA-Large audio root.
AUDIO_ROOT = PROJECT_ROOT / "data" / "raw" / "audio" / "fma_large"

# Output folder for this notebook.
OUTPUT_DIR = PROJECT_ROOT / "outputs" / "notebook49_fma_test_batch_evaluation"
OUTPUT_DIR.mkdir(parents=True, exist_ok=True)

# Final frozen artifact paths.
STRUCTURED_MODEL_PATH = MODELS_DIR /
"final_structured_multilabel_candidate150_best_model.joblib"
STRUCTURED_SCALER_PATH = MODELS_DIR /
"final_structured_multilabel_candidate150_scaler.joblib"
STRUCTURED_FEATURE_COLUMNS_PATH = PROCESSED_DIR / "structured_feature_columns.npy"
AUDIO_CNN_MODEL_PATH = MODELS_DIR /
"audio_multilabel_candidate150_expanded_final.keras"
LABEL_COLUMNS_PATH = PROCESSED_DIR /
"hybrid_multilabel_candidate150_expanded_label_columns.npy"
HYBRID_CONFIG_PATH = PROCESSED_DIR / "final_project_frozen_config.json"

# Formal evaluation source.

```

```
# "processed_expanded_test" matches Notebooks 34-36 and the saved final benchmark
tables.
# "official_fma_test" uses the raw FMA tracks.csv test split and is useful only as
an extra stress test.
EVALUATION_SPLIT_SOURCE = "processed_expanded_test"
PROCESSED_TEST_CSV = PROCESSED_DIR /
"audio_multilabel_candidate150_expanded_test.csv"

# Final frozen hybrid settings from the project.
STRUCTURED_WEIGHT = 0.10
AUDIO_WEIGHT = 0.90
STAGE1_THRESHOLD = 0.20
LOW_CONFIDENCE_THRESHOLD = 0.50

# Audio settings used by the trained audio CNN.
SR = 22050
N_MELS = 64
N_FFT = 2048
HOP_LENGTH = 1024
WINDOW_SECONDS = 15

# For formal benchmark reproduction, use the first 15-second window.
# The audio CNN was trained with librosa.load(..., duration=15), not full-song z-
score windows.
MAX_WINDOWS = 1
WINDOW_SELECTION_MODE = "first_n"

# Batch-evaluation settings.
# Use 100 for a fast validation run, then set to None for the full 1,500-row
expanded test split.
MAX_TEST_TRACKS = 100
CHECKPOINT_EVERY = 50
RESUME_FROM_CHECKPOINT = False

# Prediction behaviour.
TOP_K = 5
SAVE_ALL_LABEL_SCORES = True

print("PROJECT_ROOT:", PROJECT_ROOT)
print("PROCESSED_DIR:", PROCESSED_DIR)
print("MODELS_DIR:", MODELS_DIR)
print("AUDIO_ROOT:", AUDIO_ROOT)
print("OUTPUT_DIR:", OUTPUT_DIR)
print("EVALUATION_SPLIT_SOURCE:", EVALUATION_SPLIT_SOURCE)
print("MAX_WINDOWS:", MAX_WINDOWS)
print("WINDOW_SELECTION_MODE:", WINDOW_SELECTION_MODE)
```

```

PROJECT_ROOT: E:\SCHOOL\Masters\Capstone_FMA_Project
PROCESSED_DIR: E:\SCHOOL\Masters\Capstone_FMA_Project\data\processed
MODELS_DIR: E:\SCHOOL\Masters\Capstone_FMA_Project\models
AUDIO_ROOT: E:\SCHOOL\Masters\Capstone_FMA_Project\data\raw\audio\fma_large
OUTPUT_DIR:
E:\SCHOOL\Masters\Capstone_FMA_Project\outputs\notebook49_fma_test_batch_evaluation
EVALUATION_SPLIT_SOURCE: processed_expanded_test
MAX_WINDOWS: 1
WINDOW_SELECTION_MODE: first_n

```

In [4]:

```

# =====
# Cell 3: Utility Functions
# =====

def timestamp_now():
    return datetime.now().strftime("%Y-%m-%d %H:%M:%S")

def safe_read_json(path):
    path = Path(path)
    with open(path, "r", encoding="utf-8") as f:
        return json.load(f)

def safe_write_json(obj, path):
    path = Path(path)
    path.parent.mkdir(parents=True, exist_ok=True)
    with open(path, "w", encoding="utf-8") as f:
        json.dump(obj, f, indent=2, ensure_ascii=False)

def safe_read_csv_or_empty(path):
    path = Path(path)
    if not path.exists() or path.stat().st_size <= 2:
        return pd.DataFrame()
    try:
        return pd.read_csv(path)
    except pd.errors.EmptyDataError:
        return pd.DataFrame()

def find_first_file(root, patterns):
    """Find the first file matching any glob pattern under root."""
    root = Path(root)
    for pattern in patterns:
        matches = sorted(root.rglob(pattern))
        if matches:
            return matches[0]
    return None

def load_list_from_file(path):
    """Load a list from JSON, TXT, or CSV."""
    path = Path(path)
    suffix = path.suffix.lower()

```

```

if suffix == ".json":
    obj = safe_read_json(path)
    if isinstance(obj, list):
        return obj
    if isinstance(obj, dict):
        for key in ["label_cols", "label_columns", "labels",
"candidate_labels", "classes"]:
            if key in obj:
                return obj[key]
        # Fallback: first list-like value.
        for value in obj.values():
            if isinstance(value, list):
                return value
        raise ValueError(f"Could not find a label list inside JSON file: {path}")

if suffix in [".txt", ".lst"]:
    return [line.strip() for line in path.read_text(encoding="utf-
8").splitlines() if line.strip()]

if suffix == ".csv":
    df = pd.read_csv(path)
    if df.shape[1] == 1:
        return df.iloc[:, 0].dropna().astype(str).tolist()
    for col in ["label", "labels", "genre", "genre_title", "title",
"label_col", "label_columns"]:
        if col in df.columns:
            return df[col].dropna().astype(str).tolist()
    return df.iloc[:, 0].dropna().astype(str).tolist()

if suffix == ".npy":
    arr = np.load(path, allow_pickle=True)
    return [str(x) for x in arr.tolist()]

raise ValueError(f"Unsupported list file format: {path}")

def parse_list_cell(value):
    """Parse a string/list genre cell into a Python list."""
    if isinstance(value, list):
        return value
    if isinstance(value, tuple):
        return list(value)
    if pd.isna(value):
        return []

    text = str(value).strip()
    if text == "" or text.lower() in ["nan", "none", "null"]:
        return []

    # FMA list values are often stored like "[12, 25]".
    if text.startswith("[") and text.endswith("]"):
        try:
            parsed = ast.literal_eval(text)
            if isinstance(parsed, list):
                return parsed
            return [parsed]

```

```

    except Exception:
        pass

    # Custom multi-label separators.
    for sep in ["|", ";", ","]:
        if sep in text:
            return [item.strip() for item in text.split(sep) if item.strip()]

    return [text]

def normalise_label(label):
    return str(label).strip().lower()

def genre_id_to_label_col(genre_id):
    return f"genre_{int(genre_id)}"

def label_col_to_genre_id(label_col):
    text = str(label_col)
    if text.startswith("genre_"):
        return int(text.replace("genre_", "", 1))
    return int(text)

def fma_audio_path(track_id, audio_root=AUDIO_ROOT):
    tid = int(track_id)
    tid_str = f"{tid:06d}"
    return Path(audio_root) / tid_str[:3] / f"{tid_str}.mp3"

def flatten_columns(cols):
    """Flatten FMA MultiIndex columns to match the saved structured feature
    list."""
    if isinstance(cols, pd.MultiIndex):
        return ["_".join([str(x) for x in tup if str(x) != "nan"]).strip() for tup
in cols]
    return [str(c) for c in cols]

```

In [5]:

```

# =====
# Cell 4: Discover and Load Model Artifacts
# =====

MODEL_SEARCH_ROOTS = [
    PROJECT_ROOT / "models",
    PROJECT_ROOT / "outputs",
    PROJECT_ROOT / "notebook",
    PROJECT_ROOT,
]

def discover_artifact(manual_path, patterns, artifact_name):

```

```

    if manual_path is not None:
        p = Path(manual_path)
        if not p.exists():
            raise FileNotFoundError(f"Manual {artifact_name} path does not exist:
{p}")
        print(f"{artifact_name}: {p} [manual]")
        return p

    for root in MODEL_SEARCH_ROOTS:
        if root.exists():
            found = find_first_file(root, patterns)
            if found is not None:
                print(f"{artifact_name}: {found} [auto-discovered]")
                return found

    print(f"WARNING: {artifact_name} was not auto-discovered.")
    return None

structured_model_path = discover_artifact(
    STRUCTURED_MODEL_PATH,
    [
        "*best*structured*model*.joblib",
        "*structured*hinge*.joblib",
        "*sgd*hinge*.joblib",
        "*ovr*hinge*.joblib",
        "*structured*model*.joblib",
        "*structured*.pkl",
    ],
    "Structured model",
)

structured_scaler_path = discover_artifact(
    STRUCTURED_SCALER_PATH,
    [
        "*structured*scaler*.joblib",
        "*feature*scaler*.joblib",
        "*scaler*.joblib",
        "*scaler*.pkl",
    ],
    "Structured scaler",
)

structured_feature_columns_path = discover_artifact(
    STRUCTURED_FEATURE_COLUMNS_PATH,
    [
        "*structured*feature*columns*.numpy",
        "*structured*feature*columns*.json",
        "*feature*columns*.json",
        "*structured*features*.txt",
        "*feature_cols*.json",
    ],
    "Structured feature columns",
)

audio_cnn_model_path = discover_artifact(
    AUDIO_CNN_MODEL_PATH,

```



```

[
    "*expanded*audio*cnn*.keras",
    "*audio*cnn*tuned*.keras",
    "*audio*cnn*.keras",
    "*cnn*.keras",
    "*audio*cnn*.h5",
    "*cnn*.h5",
],
"Audio CNN model",
)

label_columns_path = discover_artifact(
    LABEL_COLUMNS_PATH,
    [
        "*hybrid_multilabel_candidate150_expanded_label_columns.npy",
        "*candidate150*label*.npy",
        "*candidate150*label*.json",
        "*candidate*label*.json",
        "*label*columns*.json",
        "*label_cols*.json",
        "*labels*.json",
        "*candidate150*label*.txt",
        "*label_cols*.txt",
    ],
    "Label columns",
)

hybrid_config_path = discover_artifact(
    HYBRID_CONFIG_PATH,
    [
        "*hybrid*config*.json",
        "*final*config*.json",
        "*frozen*config*.json",
    ],
    "Hybrid config",
)

if structured_model_path is None:
    raise FileNotFoundError("Structured model artifact not found. Set
STRUCTURED_MODEL_PATH manually in Cell 2.")
if audio_cnn_model_path is None:
    raise FileNotFoundError("Audio CNN model artifact not found. Set
AUDIO_CNN_MODEL_PATH manually in Cell 2.")
if label_columns_path is None:
    raise FileNotFoundError("Label columns file not found. Set LABEL_COLUMNS_PATH
manually in Cell 2.")

structured_model = joblib.load(structured_model_path)
structured_scaler = joblib.load(structured_scaler_path) if structured_scaler_path
else None
structured_feature_columns = load_list_from_file(structured_feature_columns_path)
if structured_feature_columns_path else None
label_cols = [str(x) for x in load_list_from_file(label_columns_path)]

audio_model = tf.keras.models.load_model(audio_cnn_model_path)

if hybrid_config_path:

```

```

try:
    cfg = safe_read_json(hybrid_config_path)
    STRUCTURED_WEIGHT = float(cfg.get("stage1_structured_weight",
    cfg.get("structured_weight", cfg.get("STRUCTURED_WEIGHT", STRUCTURED_WEIGHT))))
    AUDIO_WEIGHT = float(cfg.get("stage1_audio_weight", cfg.get("audio_weight",
    cfg.get("AUDIO_WEIGHT", AUDIO_WEIGHT))))
    STAGE1_THRESHOLD = float(cfg.get("stage1_threshold", cfg.get("threshold",
    STAGE1_THRESHOLD)))
    LOW_CONFIDENCE_THRESHOLD = float(cfg.get("low_confidence_threshold",
    LOW_CONFIDENCE_THRESHOLD))
except Exception as e:
    print("WARNING: Could not fully read hybrid config; using Cell 2
defaults.", e)

print("\nLoaded artifacts:")
print("Structured model:", type(structured_model))
print("Structured scaler:", type(structured_scaler) if structured_scaler else None)
print("Audio model input shape:", audio_model.input_shape)
print("Audio model output shape:", audio_model.output_shape)
print("Number of labels:", len(label_cols))
print("First labels:", label_cols[:5])
print("Structured feature columns:", len(structured_feature_columns) if
structured_feature_columns is not None else "all numeric FMA features")
print("Hybrid weights:", {"structured": STRUCTURED_WEIGHT, "audio": AUDIO_WEIGHT,
"threshold": STAGE1_THRESHOLD})

# Basic output-dimension check.
try:
    output_dim = int(audio_model.output_shape[-1])
    if output_dim != len(label_cols):
        print("WARNING: Audio model output dimension does not match number of label
columns.")
        print("Audio output dim:", output_dim, "Label columns:", len(label_cols))
        print("Check LABEL_COLUMNS_PATH before running the full batch.")
except Exception:
    pass

```

Structured model:

E:\SCHOOL\Masters\Capstone_FMA_Project\models\final_structured_multilabel_candidate15
0_best_model.joblib [manual]

Structured scaler:

E:\SCHOOL\Masters\Capstone_FMA_Project\models\final_structured_multilabel_candidate15
0_scaler.joblib [manual]

Structured feature columns:

E:\SCHOOL\Masters\Capstone_FMA_Project\data\processed\structured_feature_columns.npy
[manual]

Audio CNN model:

E:\SCHOOL\Masters\Capstone_FMA_Project\models\audio_multilabel_candidate150_expanded_
final.keras [manual]

Label columns:

E:\SCHOOL\Masters\Capstone_FMA_Project\data\processed\hybrid_multilabel_candidate150_
expanded_label_columns.npy [manual]

Hybrid config:

E:\SCHOOL\Masters\Capstone_FMA_Project\data\processed\final_project_frozen_config.jso
n [manual]

Loaded artifacts:

Structured model: <class 'sklearn.multiclass.OneVsRestClassifier'>

Structured scaler: <class 'sklearn.preprocessing._data.StandardScaler'>

Audio model input shape: (None, 64, 324, 1)

Audio model output shape: (None, 150)

Number of labels: 150

First labels: ['genre_1', 'genre_2', 'genre_3', 'genre_4', 'genre_5']

Structured feature columns: 518

Hybrid weights: {'structured': 0.1, 'audio': 0.9, 'threshold': 0.2}

In [6]:

```
# =====
# Cell 5: Load FMA Metadata, Genre Hierarchy, and Features
# =====

for p in [TRACKS_CSV, GENRES_CSV, FEATURES_CSV]:
    if not Path(p).exists():
        raise FileNotFoundError(f"Missing required file: {p}")

# FMA tracks.csv normally has two header rows.
tracks = pd.read_csv(TRACKS_CSV, index_col=0, header=[0, 1])
tracks.index = tracks.index.astype(int)
print("tracks shape:", tracks.shape)

# genres.csv is a standard CSV.
genres = pd.read_csv(GENRES_CSV)
if "genre_id" not in genres.columns:
    genres = genres.reset_index().rename(columns={"index": "genre_id"})
genres["genre_id"] = genres["genre_id"].astype(int)

genre_id_to_title = dict(zip(genres["genre_id"], genres["title"].astype(str)))
genre_title_to_id = {v: k for k, v in genre_id_to_title.items()}
print("genres shape:", genres.shape)

# FMA features.csv normally has three header rows.
features = pd.read_csv(FEATURES_CSV, index_col=0, header=[0, 1, 2])
features.index = features.index.astype(int)
features.columns = flatten_columns(features.columns)
print("features shape:", features.shape)

# Identify split and genre columns robustly.
def get_track_col(primary, secondary):
    col = (primary, secondary)
    if col in tracks.columns:
        return col
    # Fallback search.
    for c in tracks.columns:
        if len(c) >= 2 and str(c[0]).lower() == primary.lower() and
str(c[1]).lower() == secondary.lower():
            return c
    raise KeyError(f"Could not find tracks column: ({primary}, {secondary})")

split_col = get_track_col("set", "split")
subset_col = get_track_col("set", "subset") if ("set", "subset") in tracks.columns
else None
track_genres_col = get_track_col("track", "genres")
```

```

track_genres_all_col = get_track_col("track", "genres_all")

print("split column:", split_col)
print("genres column:", track_genres_col)
print("genres_all column:", track_genres_all_col)

```

```

tracks shape: (106574, 52)
genres shape: (163, 5)
features shape: (106574, 518)
split column: ('set', 'split')
genres column: ('track', 'genres')
genres_all column: ('track', 'genres_all')

```

In [7]:

```

# =====
# Cell 6: Build the Test-Split Evaluation Table
# =====

# The frozen candidate label space is encoded as genre IDs, e.g. genre_21.
# The default processed-expanded split matches the split used by Notebooks 34-36.
TRUE_LABEL_SOURCE = "genres_all" # used only when EVALUATION_SPLIT_SOURCE ==
"official_fma_test"

def ids_to_label_cols(values):
    ids = parse_list_cell(values)
    labels = []
    for x in ids:
        try:
            gid = int(x)
            labels.append(genre_id_to_label_col(gid))
        except Exception:
            text = str(x).strip()
            if text.startswith("genre_"):
                labels.append(text)
            elif text in genre_title_to_id:
                labels.append(genre_id_to_label_col(genre_title_to_id[text]))
    return list(dict.fromkeys(labels))

def resolve_audio_path(path_value, track_id=None):
    text = str(path_value).strip()
    if text and text.lower() not in ["nan", "none", "null"]:
        p = Path(text)
        if p.is_absolute():
            return str(p)
        # Processed split files store paths like
        ../data/raw/audio/fma_large\000\000568.mp3.
        normalized = text.replace("\\", "/")
        if normalized.startswith("../"):
            return str((PROJECT_ROOT / normalized.replace("../", "", 1)).resolve())
        return str((PROJECT_ROOT / normalized).resolve())
    if track_id is not None:
        return str(fma_audio_path(track_id))
    return ""

```

```

label_cols_norm = [normalise_label(x) for x in label_cols]
label_norm_to_raw = dict(zip(label_cols_norm, label_cols))

def keep_candidate_true_labels(true_labels):
    true_set = set(normalise_label(x) for x in true_labels)
    return [label_norm_to_raw[lbl] for lbl in label_cols_norm if lbl in true_set]

if EVALUATION_SPLIT_SOURCE == "processed_expanded_test":
    if not PROCESSED_TEST_CSV.exists():
        raise FileNotFoundError(f"Missing processed expanded test split:
{PROCESSED_TEST_CSV}")

    processed_test_df = pd.read_csv(PROCESSED_TEST_CSV)
    missing_label_cols = [c for c in label_cols if c not in
processed_test_df.columns]
    if missing_label_cols:
        raise ValueError(f"Processed test split is missing label columns. Example:
{missing_label_cols[:10]}")

    eval_df = processed_test_df.copy()
    eval_df["track_id"] = eval_df["track_id"].astype(int)
    eval_df["split"] = eval_df.get("split", "test")
    eval_df["audio_path"] = eval_df.apply(
        lambda row: resolve_audio_path(row.get("audio_path", ""), row["track_id"]),
        axis=1,
    )
    eval_df["audio_exists"] = eval_df["audio_path"].apply(lambda p:
Path(p).exists())
    eval_df["true_candidate_labels"] = eval_df[label_cols].apply(
        lambda row: [label for label in label_cols if int(row[label]) == 1],
        axis=1,
    )
    eval_df["true_labels"] = eval_df["true_candidate_labels"]
    eval_df["features_available"] = eval_df["track_id"].isin(features.index)

    test_df = eval_df[
        (eval_df["audio_exists"])
        & (eval_df["features_available"])
        & (eval_df["true_candidate_labels"].apply(len) > 0)
    ].copy()

elif EVALUATION_SPLIT_SOURCE == "official_fma_test":
    selected_genre_col = track_genres_all_col if TRUE_LABEL_SOURCE == "genres_all"
else track_genres_col

    eval_df = pd.DataFrame(index=tracks.index)
    eval_df["track_id"] = eval_df.index.astype(int)
    eval_df["split"] = tracks[split_col].astype(str).values
    eval_df["true_labels"] =
tracks[selected_genre_col].apply(ids_to_label_cols).values
    eval_df["audio_path"] = eval_df["track_id"].apply(lambda x:
str(fma_audio_path(x)))
    eval_df["audio_exists"] = eval_df["audio_path"].apply(lambda p:

```

```

Path(p).exists())
    eval_df["true_candidate_labels"] =
eval_df["true_labels"].apply(keep_candidate_true_labels)
    eval_df["features_available"] = eval_df["track_id"].isin(features.index)

    test_df = eval_df[
        (eval_df["split"].str.lower() == "test")
        & (eval_df["audio_exists"])
        & (eval_df["features_available"])
        & (eval_df["true_candidate_labels"].apply(len) > 0)
    ].copy()

else:
    raise ValueError("EVALUATION_SPLIT_SOURCE must be 'processed_expanded_test' or
'official_fma_test'.")

test_df = test_df.reset_index(drop=True)

if MAX_TEST_TRACKS is not None:
    test_df = test_df.head(int(MAX_TEST_TRACKS)).copy()

print("Evaluation source:", EVALUATION_SPLIT_SOURCE)
print("Candidate records before MAX_TEST_TRACKS:", len(eval_df))
print("Test records selected:", len(test_df))
print("Records with audio:", int(test_df["audio_exists"].sum()) if len(test_df)
else 0)
print("Records with features:", int(test_df["features_available"].sum()) if
len(test_df) else 0)
print(test_df[["track_id", "split", "audio_exists", "features_available",
"true_candidate_labels"]].head())

test_df.to_csv(OUTPUT_DIR / "fma_test_records_selected.csv", index=False)

```

Evaluation source: processed_expanded_test

Candidate records before MAX_TEST_TRACKS: 1500

Test records selected: 100

Records with audio: 100

Records with features: 100

	track_id	split	audio_exists	features_available	true_candidate_labels
0	568	test	True	True	[genre_12]
1	982	test	True	True	[genre_30, genre_38]
2	984	test	True	True	[genre_30, genre_38]
3	988	test	True	True	[genre_30, genre_38]
4	1019	test	True	True	[genre_32, genre_38]

In [8]:

```

# =====
# Cell 7: Audio Windowing and Mel-Spectrogram Helpers
# =====

MAX_FRAMES = int(math.ceil((WINDOW_SECONDS * SR) / HOP_LENGTH)) + 1
WINDOW_SAMPLES = int(WINDOW_SECONDS * SR)

def select_window_starts(num_samples, window_samples=WINDOW_SAMPLES,

```

```

max_windows=MAX_WINDOWS, mode=WINDOW_SELECTION_MODE):
    """Return sample start positions for windows."""
    if num_samples <= window_samples:
        return [0]

    max_start = num_samples - window_samples

    if max_windows <= 1:
        return [0]

    if mode == "first_n":
        starts = []
        current = 0
        while current <= max_start and len(starts) < max_windows:
            starts.append(int(current))
            current += window_samples
        return starts if starts else [0]

    if mode == "evenly_spaced":
        n_windows = min(max_windows, max(2, math.ceil(num_samples /
window_samples)))
        starts = np.linspace(0, max_start, num=n_windows)
        return sorted(list(dict.fromkeys([int(x) for x in starts]))[:max_windows])

    raise ValueError("WINDOW_SELECTION_MODE must be 'first_n' or 'evenly_spaced'.")

def pad_or_trim_frames(mel_db, max_frames=MAX_FRAMES):
    """Pad or trim a Mel spectrogram to fixed time frames."""
    if mel_db.shape[1] < max_frames:
        pad_width = max_frames - mel_db.shape[1]
        mel_db = np.pad(mel_db, ((0, 0), (0, pad_width)), mode="constant")
    elif mel_db.shape[1] > max_frames:
        mel_db = mel_db[:, :max_frames]
    return mel_db

def segment_to_training_mel(segment):
    """Match Notebook 34 audio CNN preprocessing: dB Mel clipped/scaled to [0,
1]."""
    mel = librosa.feature.melspectrogram(
        y=segment,
        sr=SR,
        n_fft=N_FFT,
        hop_length=HOP_LENGTH,
        n_mels=N_MELS,
        power=2.0,
    )
    mel_db = librosa.power_to_db(mel, ref=np.max)
    mel_db = np.clip((mel_db + 80.0) / 80.0, 0.0, 1.0)
    mel_db = pad_or_trim_frames(mel_db)
    return mel_db.astype(np.float32)

def audio_file_to_window_batch(audio_path):
    """Load an audio file and return training-consistent Mel-spectrogram
windows."""

```

```

if MAX_WINDOWS <= 1:
    y, _ = librosa.load(audio_path, sr=SR, mono=True, duration=WINDOW_SECONDS)
else:
    y, _ = librosa.load(audio_path, sr=SR, mono=True)

if y is None or len(y) == 0:
    raise ValueError("Loaded audio has zero samples.")

if len(y) < WINDOW_SAMPLES:
    y = np.pad(y, (0, WINDOW_SAMPLES - len(y)), mode="constant")

starts = select_window_starts(len(y))
windows = []

for start in starts:
    segment = y[start:start + WINDOW_SAMPLES]
    if len(segment) < WINDOW_SAMPLES:
        segment = np.pad(segment, (0, WINDOW_SAMPLES - len(segment)),
mode="constant")
        windows.append(segment_to_training_mel(segment))

X = np.array(windows, dtype=np.float32)

if len(audio_model.input_shape) == 4:
    X = X[..., np.newaxis]

return X, starts

print("Audio settings:")
print({
    "SR": SR,
    "N_MELS": N_MELS,
    "N_FFT": N_FFT,
    "HOP_LENGTH": HOP_LENGTH,
    "WINDOW_SECONDS": WINDOW_SECONDS,
    "MAX_FRAMES": MAX_FRAMES,
    "MAX_WINDOWS": MAX_WINDOWS,
    "WINDOW_SELECTION_MODE": WINDOW_SELECTION_MODE,
    "mel_scaling": "Notebook 34 training-compatible [0, 1] dB Mel scaling",
})

```

Audio settings:

```
{'SR': 22050, 'N_MELS': 64, 'N_FFT': 2048, 'HOP_LENGTH': 1024, 'WINDOW_SECONDS': 15,
'MAX_FRAMES': 324, 'MAX_WINDOWS': 1, 'WINDOW_SELECTION_MODE': 'first_n',
'mel_scaling': 'Notebook 34 training-compatible [0, 1] dB Mel scaling'}
```

In [9]:

```

# =====
# Cell 8: Structured and Hybrid Prediction Helpers
# =====

def get_structured_features_for_track(track_id):
    """Return one-row structured feature matrix for a track ID."""
    if int(track_id) not in features.index:
        raise KeyError(f"Track ID not found in features.csv: {track_id}")

```



```

row = features.loc[[int(track_id)]].copy()

if structured_feature_columns is not None:
    missing = [c for c in structured_feature_columns if c not in row.columns]
    if missing:
        raise ValueError(
            f"Missing {len(missing)} structured feature columns. Example
missing: {missing[:10]}"
        )
    row = row[structured_feature_columns]

row = row.replace([np.inf, -np.inf], np.nan).fillna(0.0)
X = row.values.astype(np.float32)

if structured_scaler is not None:
    X = structured_scaler.transform(X)

return X

def structured_predict_scores(track_id):
    """Predict structured scores/probabilities for one track."""
    X = get_structured_features_for_track(track_id)

    if hasattr(structured_model, "predict_proba"):
        proba = structured_model.predict_proba(X)
        # OneVsRestClassifier predict_proba returns array shape (n_samples,
n_labels)
        if isinstance(proba, list):
            proba = np.vstack([p[:, 1] for p in proba]).T
            scores = np.asarray(proba)[0]
        elif hasattr(structured_model, "decision_function"):
            decision = structured_model.decision_function(X)
            scores = expit(np.asarray(decision)[0])
        else:
            pred = structured_model.predict(X)
            scores = np.asarray(pred)[0].astype(float)

    return np.asarray(scores, dtype=np.float32)

def audio_predict_scores(audio_path):
    """Predict averaged audio CNN probabilities for one audio file."""
    X, starts = audio_file_to_window_batch(audio_path)
    preds = audio_model.predict(X, verbose=0)
    preds = np.asarray(preds, dtype=np.float32)
    if preds.ndim == 1:
        preds = preds.reshape(1, -1)
    mean_scores = preds.mean(axis=0)
    return mean_scores, starts, preds

def final_hybrid_predict(track_id, audio_path):
    """Run structured + audio + hybrid prediction for one track."""
    s_scores = structured_predict_scores(track_id)
    a_scores, starts, window_scores = audio_predict_scores(audio_path)

```

```

# Dimension safety.
n = len(label_cols)
s_scores = np.asarray(s_scores[:n], dtype=np.float32)
a_scores = np.asarray(a_scores[:n], dtype=np.float32)

hybrid_scores = (STRUCTURED_WEIGHT * s_scores) + (AUDIO_WEIGHT * a_scores)

order = np.argsort(hybrid_scores)[::-1]
top_labels = [label_cols[i] for i in order[:TOP_K]]
top_scores = [float(hybrid_scores[i]) for i in order[:TOP_K]]

predicted_labels = [label_cols[i] for i, score in enumerate(hybrid_scores) if
score >= STAGE1_THRESHOLD]
low_confidence = bool(float(hybrid_scores[order[0]]) <
LOW_CONFIDENCE_THRESHOLD)

return {
    "structured_scores": s_scores,
    "audio_scores": a_scores,
    "hybrid_scores": hybrid_scores,
    "top_labels": top_labels,
    "top_scores": top_scores,
    "predicted_labels": predicted_labels,
    "low_confidence": low_confidence,
    "window_starts": starts,
    "num_windows": len(starts),
}

print("Prediction helpers ready.")

```

Prediction helpers ready.

In [10]:

```

# =====
# Cell 9: Batch Prediction Loop with Checkpointing
# =====

checkpoint_path = OUTPUT_DIR / "fma_test_predictions_checkpoint.csv"
runtime_log_path = OUTPUT_DIR / "fma_test_runtime_log.csv"

completed_track_ids = set()
rows = []
runtime_rows = []

if RESUME_FROM_CHECKPOINT and checkpoint_path.exists():
    checkpoint_df = safe_read_csv_or_empty(checkpoint_path)
    if checkpoint_df.empty:
        print(f"Checkpoint exists but is empty; starting a fresh batch:
{checkpoint_path}")
    elif "track_id" in checkpoint_df.columns:
        completed_track_ids = set(checkpoint_df["track_id"].astype(int).tolist())
        rows = checkpoint_df.to_dict("records")
        print(f"Resuming from checkpoint with {len(completed_track_ids)} completed
tracks.")

```

```

else:
    print(f"Checkpoint has no track_id column; starting a fresh batch:
{checkpoint_path}")

start_time_all = time.time()

def save_checkpoint(rows):
    pd.DataFrame(rows).to_csv(checkpoint_path, index=False)

for idx, rec in test_df.iterrows():
    track_id = int(rec["track_id"])
    if track_id in completed_track_ids:
        continue

    audio_path = rec["audio_path"]
    true_labels = rec["true_candidate_labels"]

    t0 = time.time()
    status = "ok"
    error_msg = ""

    try:
        pred = final_hybrid_predict(track_id, audio_path)
        hybrid_scores = pred["hybrid_scores"]

        row = {
            "track_id": track_id,
            "audio_path": audio_path,
            "true_labels": "|".join(true_labels),
            "predicted_labels": "|".join(pred["predicted_labels"]),
            "top1_label": pred["top_labels"][0] if pred["top_labels"] else "",
            "top1_score": pred["top_scores"][0] if pred["top_scores"] else np.nan,
            "topk_labels": "|".join(pred["top_labels"]),
            "topk_scores": "|".join([f"{x:.6f}" for x in pred["top_scores"]]),
            "low_confidence": pred["low_confidence"],
            "num_windows": pred["num_windows"],
            "window_starts": "|".join([str(x) for x in pred["window_starts"]]),
            "status": status,
            "error": error_msg,
        }

        if SAVE_ALL_LABEL_SCORES:
            for label, score in zip(label_cols, hybrid_scores):
                safe_label = label.replace(" ", "_").replace("/", "_").replace("-",
"_)

                row[f"score_{safe_label}"] = float(score)

    except Exception as e:
        status = "error"
        error_msg = str(e)
        row = {
            "track_id": track_id,
            "audio_path": audio_path,
            "true_labels": "|".join(true_labels),
            "predicted_labels": "",
            "top1_label": "",
            "top1_score": np.nan,

```

```

        "topk_labels": "",
        "topk_scores": "",
        "low_confidence": True,
        "num_windows": 0,
        "window_starts": "",
        "status": status,
        "error": error_msg,
    }

    elapsed = time.time() - t0
    rows.append(row)
    completed_track_ids.add(track_id)
    runtime_rows.append({
        "timestamp": timestamp_now(),
        "track_id": track_id,
        "status": status,
        "seconds": elapsed,
        "error": error_msg,
    })

    if len(rows) % CHECKPOINT_EVERY == 0:
        save_checkpoint(rows)
        pd.DataFrame(runtime_rows).to_csv(runtime_log_path, index=False)
        print(f"Checkpoint saved: {len(rows)} rows | latest track {track_id} | "
              f"status={status} | {elapsed:.2f}s")

    save_checkpoint(rows)
    pd.DataFrame(runtime_rows).to_csv(runtime_log_path, index=False)

    elapsed_all = time.time() - start_time_all
    print(f"Batch prediction completed or checkpointed. Total rows: {len(rows)}")
    print(f"Elapsed time this run: {elapsed_all/60:.2f} minutes")
    print("Checkpoint:", checkpoint_path)

```

Checkpoint saved: 50 rows | latest track 9256 | status=ok | 0.16s

Checkpoint saved: 100 rows | latest track 13812 | status=ok | 0.16s

Batch prediction completed or checkpointed. Total rows: 100

Elapsed time this run: 0.76 minutes

Checkpoint:

E:\SCHOOL\Masters\Capstone_FMA_Project\outputs\notebook49_fma_test_batch_evaluation\ma_test_predictions_checkpoint.csv

In [11]:

```

# =====
# Cell 10: Build Evaluation Matrices
# =====

pred_df = safe_read_csv_or_empty(checkpoint_path)

if pred_df.empty:
    raise ValueError("Prediction checkpoint is empty. Re-run Cell 9 after Cell 6
has selected records.")

ok_df = pred_df[pred_df["status"].astype(str).str.lower() == "ok"].copy()
error_df = pred_df[pred_df["status"].astype(str).str.lower() != "ok"].copy()

```

```

print("Total predictions:", len(pred_df))
print("Successful predictions:", len(ok_df))
print("Errors:", len(error_df))

# Score columns created in Cell 9.
score_cols = [c for c in ok_df.columns if c.startswith("score__")]

if not score_cols:
    raise ValueError("No score columns found. Re-run Cell 9 with
SAVE_ALL_LABEL_SCORES=True.")

# Recover score-label mapping from label_cols using same safe naming rule.
label_to_score_col = {}
for label in label_cols:
    safe_label = label.replace(" ", "_").replace("/", "_").replace("-", "_")
    col = f"score__{safe_label}"
    if col in ok_df.columns:
        label_to_score_col[label] = col

available_labels = [label for label in label_cols if label in label_to_score_col]
print("Available labels for metrics:", len(available_labels))

# y_score matrix.
y_score = ok_df[[label_to_score_col[label] for label in
available_labels]].values.astype(float)
y_pred = (y_score >= STAGE1_THRESHOLD).astype(int)

# y_true matrix from true labels.
label_norm_to_idx = {normalise_label(label): i for i, label in
enumerate(available_labels)}

y_true = np.zeros_like(y_pred, dtype=int)
for row_idx, labels_text in enumerate(ok_df["true_labels"].fillna("")):
    labels = [x.strip() for x in str(labels_text).split("|") if x.strip()]
    for label in labels:
        idx = label_norm_to_idx.get(normalise_label(label))
        if idx is not None:
            y_true[row_idx, idx] = 1

print("y_true shape:", y_true.shape)
print("y_pred shape:", y_pred.shape)
print("y_score shape:", y_score.shape)
print("Positive true labels:", int(y_true.sum()))
print("Positive predicted labels:", int(y_pred.sum()))

```

```

Total predictions: 100
Successful predictions: 100
Errors: 0
Available labels for metrics: 150
y_true shape: (100, 150)
y_pred shape: (100, 150)
y_score shape: (100, 150)
Positive true labels: 295
Positive predicted labels: 447

```

In [12]:

```

# =====
# Cell 11: Calculate Metrics and Save Outputs
# =====

metrics_rows = []

for avg in ["micro", "macro", "samples"]:
    p, r, f1, support = precision_recall_fscore_support(
        y_true,
        y_pred,
        average=avg,
        zero_division=0,
    )
    metrics_rows.append({
        "metric_group": avg,
        "precision": float(p),
        "recall": float(r),
        "f1": float(f1),
        "support": int(y_true.sum()) if avg != "samples" else len(y_true),
    })

metrics_summary = pd.DataFrame(metrics_rows)

# Additional metrics.
additional_metrics = {
    "hamming_loss": float(hamming_loss(y_true, y_pred)),
    "jaccard_samples": float(jaccard_score(y_true, y_pred, average="samples",
zero_division=0)),
    "num_successful_tracks": int(len(ok_df)),
    "num_error_tracks": int(len(error_df)),
    "num_labels": int(len(available_labels)),
    "stage1_threshold": float(STAGE1_THRESHOLD),
    "structured_weight": float(STRUCTURED_WEIGHT),
    "audio_weight": float(AUDIO_WEIGHT),
    "low_confidence_threshold": float(LOW_CONFIDENCE_THRESHOLD),
    "low_confidence_count":
int(ok_df["low_confidence"].astype(str).str.lower().isin(["true", "1"]).sum()),
}

# Top-K hit-any rates: useful for recommendation-style interpretation.
def topk_hit(labels_text, topk_text, k):
    true_set = set(normalise_label(x) for x in str(labels_text).split("|") if
str(x).strip())
    top_list = [x.strip() for x in str(topk_text).split("|") if x.strip()][:k]
    top_set = set(normalise_label(x) for x in top_list)
    return int(len(true_set.intersection(top_set)) > 0)

for k in [1, 3, 5]:
    hits = [topk_hit(t, p, k) for t, p in zip(ok_df["true_labels"],
ok_df["topk_labels"])]
    hit_rate = float(np.mean(hits)) if hits else 0.0
    additional_metrics[f"top{k}_hit_rate"] = hit_rate
    additional_metrics[f"hit_any_at_{k}"] = hit_rate

```

```

# Per-label metrics.
per_label_p, per_label_r, per_label_f1, per_label_support =
precision_recall_fscore_support(
    y_true,
    y_pred,
    average=None,
    zero_division=0,
)

per_label_df = pd.DataFrame({
    "label": available_labels,
    "precision": per_label_p,
    "recall": per_label_r,
    "f1": per_label_f1,
    "support": per_label_support,
})
per_label_df = per_label_df.sort_values(["support", "f1"], ascending=[False,
False])

# Top-K Long format.
topk_rows = []
for _, row in ok_df.iterrows():
    true_set = set(normalise_label(x) for x in str(row["true_labels"]).split("|"))
    if x.strip()
    labels = [x.strip() for x in str(row["topk_labels"]).split("|") if x.strip()]
    scores = [float(x) for x in str(row["topk_scores"]).split("|") if
str(x).strip()]
    for rank, (label, score) in enumerate(zip(labels, scores), start=1):
        topk_rows.append({
            "track_id": row["track_id"],
            "rank": rank,
            "label": label,
            "score": score,
            "is_true_label": normalise_label(label) in true_set,
            "true_labels": row["true_labels"],
        })

topk_df = pd.DataFrame(topk_rows)

# Error and Low-confidence cases.
low_conf_df = ok_df[ok_df["low_confidence"].astype(str).str.lower().isin(["true",
"1"])]

# Save outputs.
pred_summary_cols = [
    "track_id", "audio_path", "true_labels", "predicted_labels", "top1_label",
    "top1_score",
    "topk_labels", "topk_scores", "low_confidence", "num_windows", "window_starts",
    "status", "error"
]

ok_df[pred_summary_cols].to_csv(OUTPUT_DIR / "fma_test_prediction_summary.csv",
index=False)
topk_df.to_csv(OUTPUT_DIR / "fma_test_topk_predictions.csv", index=False)
metrics_summary.to_csv(OUTPUT_DIR / "fma_test_metrics_summary.csv", index=False)
per_label_df.to_csv(OUTPUT_DIR / "fma_test_per_label_metrics.csv", index=False)
low_conf_df[pred_summary_cols].to_csv(OUTPUT_DIR /

```

```

"fma_test_low_confidence_cases.csv", index=False)
error_df.to_csv(OUTPUT_DIR / "fma_test_error_cases.csv", index=False)

if SAVE_ALL_LABEL_SCORES:
    ok_df[["track_id"] + [label_to_score_col[label] for label in
available_labels]].to_csv(
        OUTPUT_DIR / "fma_test_all_label_scores.csv",
        index=False,
    )

summary_json = {
    "created_at": timestamp_now(),
    "notebook": "49_final_fma_test_split_batch_evaluation.ipynb",
    "true_label_source": TRUE_LABEL_SOURCE,
    "metrics_summary": metrics_summary.to_dict(orient="records"),
    "additional_metrics": additional_metrics,
    "output_files": {
        "prediction_summary": str(OUTPUT_DIR / "fma_test_prediction_summary.csv"),
        "topk_predictions": str(OUTPUT_DIR / "fma_test_topk_predictions.csv"),
        "all_label_scores": str(OUTPUT_DIR / "fma_test_all_label_scores.csv"),
        "metrics_summary": str(OUTPUT_DIR / "fma_test_metrics_summary.csv"),
        "per_label_metrics": str(OUTPUT_DIR / "fma_test_per_label_metrics.csv"),
        "low_confidence_cases": str(OUTPUT_DIR /
"fma_test_low_confidence_cases.csv"),
        "error_cases": str(OUTPUT_DIR / "fma_test_error_cases.csv"),
        "runtime_log": str(runtime_log_path),
    },
}
safe_write_json(summary_json, OUTPUT_DIR / "fma_test_summary.json")

print("\nMetrics summary:")
display(metrics_summary)

print("\nAdditional metrics:")
for k, v in additional_metrics.items():
    print(f"{k}: {v}")

print("\nTop per-label metrics by support:")
display(per_label_df.head(20))

print("\nSaved outputs to:", OUTPUT_DIR)

```

Metrics summary:

	metric_group	precision	recall	f1	support
0	micro	0.373602	0.566102	0.450135	295
1	macro	0.047072	0.077065	0.051506	295
2	samples	0.388619	0.645476	0.445483	100

Additional metrics:

hamming_loss: 0.0272
 jaccard_samples: 0.30071031746031746
 num_successful_tracks: 100
 num_error_tracks: 0
 num_labels: 150
 stage1_threshold: 0.2
 structured_weight: 0.1
 audio_weight: 0.9
 low_confidence_threshold: 0.5
 low_confidence_count: 12
 top1_hit_rate: 0.72
 hit_any_at_1: 0.72
 top3_hit_rate: 0.98
 hit_any_at_3: 0.98
 top5_hit_rate: 0.99
 hit_any_at_5: 0.99

Top per-label metrics by support:

	label	precision	recall	f1	support
31	genre_38	0.580000	1.000000	0.734177	58
11	genre_12	0.466667	0.945946	0.625000	37
14	genre_15	0.366667	0.785714	0.500000	28
27	genre_32	0.535714	0.600000	0.566038	25
24	genre_27	0.250000	0.153846	0.190476	13
25	genre_30	0.500000	0.181818	0.266667	11
9	genre_10	0.263158	0.555556	0.357143	9
16	genre_17	0.388889	0.875000	0.538462	8
65	genre_103	0.500000	0.428571	0.461538	7
99	genre_250	0.333333	0.500000	0.400000	6
22	genre_25	0.157895	0.500000	0.240000	6
0	genre_1	0.135135	0.833333	0.232558	6
48	genre_76	0.000000	0.000000	0.000000	6
72	genre_125	0.000000	0.000000	0.000000	6
85	genre_181	0.000000	0.000000	0.000000	6
20	genre_21	0.083333	0.200000	0.117647	5
32	genre_41	0.000000	0.000000	0.000000	5
62	genre_100	0.000000	0.000000	0.000000	5
149	genre_1235	0.133333	0.500000	0.210526	4
33	genre_42	0.000000	0.000000	0.000000	4

Saved outputs to:

E:\SCHOOL\Masters\Capstone_FMA_Project\outputs\notebook49_fma_test_batch_evaluation

In [13]:

```
# =====
# Cell 12: Quick Review Tables for Documentation
# =====

print("Best-supported labels:")
display(per_label_df.sort_values("support", ascending=False).head(15))

print("Best F1 labels with at least 20 support:")
display(per_label_df[per_label_df["support"] >= 20].sort_values("f1",
ascending=False).head(15))

print("Weak labels with at least 20 support:")
display(per_label_df[per_label_df["support"] >= 20].sort_values("f1",
ascending=True).head(15))

print("Low confidence examples:")
display(low_conf_df[pred_summary_cols].head(15))

print("Top-K examples:")
display(topk_df.head(25))
```

Best-supported labels:

	label	precision	recall	f1	support
31	genre_38	0.580000	1.000000	0.734177	58
11	genre_12	0.466667	0.945946	0.625000	37
14	genre_15	0.366667	0.785714	0.500000	28
27	genre_32	0.535714	0.600000	0.566038	25
24	genre_27	0.250000	0.153846	0.190476	13
25	genre_30	0.500000	0.181818	0.266667	11
9	genre_10	0.263158	0.555556	0.357143	9
16	genre_17	0.388889	0.875000	0.538462	8
65	genre_103	0.500000	0.428571	0.461538	7
99	genre_250	0.333333	0.500000	0.400000	6
22	genre_25	0.157895	0.500000	0.240000	6
0	genre_1	0.135135	0.833333	0.232558	6
48	genre_76	0.000000	0.000000	0.000000	6
72	genre_125	0.000000	0.000000	0.000000	6
85	genre_181	0.000000	0.000000	0.000000	6

Best F1 labels with at least 20 support:

	label	precision	recall	f1	support
31	genre_38	0.580000	1.000000	0.734177	58
11	genre_12	0.466667	0.945946	0.625000	37
27	genre_32	0.535714	0.600000	0.566038	25
14	genre_15	0.366667	0.785714	0.500000	28

Weak labels with at least 20 support:

	label	precision	recall	f1	support
14	genre_15	0.366667	0.785714	0.500000	28
27	genre_32	0.535714	0.600000	0.566038	25
11	genre_12	0.466667	0.945946	0.625000	37
31	genre_38	0.580000	1.000000	0.734177	58

Low confidence examples:

	track_id	audio_path	
18	1867	E:\SCHOOL\Masters\Capstone_FMA_Project\data\ra...	ge
20	1873	E:\SCHOOL\Masters\Capstone_FMA_Project\data\ra...	ge
35	5279	E:\SCHOOL\Masters\Capstone_FMA_Project\data\ra...	
43	7463	E:\SCHOOL\Masters\Capstone_FMA_Project\data\ra...	genre_12 genre_21 genre_25 genre_...
48	9090	E:\SCHOOL\Masters\Capstone_FMA_Project\data\ra...	gen
50	9262	E:\SCHOOL\Masters\Capstone_FMA_Project\data\ra...	genre_10 genre_17 genre...
52	9268	E:\SCHOOL\Masters\Capstone_FMA_Project\data\ra...	genre_10 genre_17 genre...
54	9349	E:\SCHOOL\Masters\Capstone_FMA_Project\data\ra...	gen
62	10505	E:\SCHOOL\Masters\Capstone_FMA_Project\data\ra...	g
66	10546	E:\SCHOOL\Masters\Capstone_FMA_Project\data\ra...	genre
79	11615	E:\SCHOOL\Masters\Capstone_FMA_Project\data\ra...	
88	13160	E:\SCHOOL\Masters\Capstone_FMA_Project\data\ra...	genre_12 genre_15 genre_27 genre_...



Top-K examples:

	track_id	rank	label	score	is_true_label	true_labels
0	568	1	genre_12	0.679192	True	genre_12
1	568	2	genre_38	0.368890	False	genre_12
2	568	3	genre_25	0.348920	False	genre_12
3	568	4	genre_15	0.235148	False	genre_12
4	568	5	genre_10	0.228779	False	genre_12
5	982	1	genre_38	0.735213	True	genre_30 genre_38
6	982	2	genre_15	0.285773	False	genre_30 genre_38
7	982	3	genre_32	0.241977	False	genre_30 genre_38
8	982	4	genre_12	0.237230	False	genre_30 genre_38
9	982	5	genre_1	0.185203	False	genre_30 genre_38
10	984	1	genre_38	0.741622	True	genre_30 genre_38
11	984	2	genre_12	0.261479	False	genre_30 genre_38
12	984	3	genre_15	0.181492	False	genre_30 genre_38
13	984	4	genre_32	0.178634	False	genre_30 genre_38
14	984	5	genre_1	0.176490	False	genre_30 genre_38
15	988	1	genre_38	0.594619	True	genre_30 genre_38
16	988	2	genre_12	0.394691	False	genre_30 genre_38
17	988	3	genre_32	0.213852	False	genre_30 genre_38
18	988	4	genre_15	0.209278	False	genre_30 genre_38
19	988	5	genre_25	0.203839	False	genre_30 genre_38
20	1019	1	genre_38	0.778958	True	genre_32 genre_38
21	1019	2	genre_15	0.294912	False	genre_32 genre_38
22	1019	3	genre_12	0.246815	False	genre_32 genre_38
23	1019	4	genre_1235	0.240525	False	genre_32 genre_38
24	1019	5	genre_32	0.239740	True	genre_32 genre_38

Interpretation Guide for the Final Report

Use this wording when reporting this notebook in the final documentation:

Notebook 49 performed formal batch evaluation on the processed expanded FMA candidate-150 held-out test split. The trained structured model, audio CNN, and hybrid fusion configuration were applied to validated test records

using the same first-15-second audio preprocessing used during training. Predictions were compared against the aligned multi-label ground truth, and performance was summarised using Micro F1, Macro F1, Samples F1, precision, recall, Top-K hit-any rates, low-confidence counts, and per-label metrics.

Important interpretation:

- **Micro F1** is the main overall multi-label performance measure.
- **Macro F1** is expected to be lower because it gives equal weight to rare labels.
- **Samples F1** shows track-level multi-label prediction quality.
- **Top-K hit-any rates** support the recommendation-style claim: even when the exact top genre is not rank 1, the correct genre may still appear in the top 3 or top 5.
- **Low-confidence cases** identify uncertain predictions and possible label/audio ambiguity.

Notebook 50 handles external-folder full-song windowed testing separately. Those results are useful for deployment analysis, but they should not replace the formal FMA benchmark.